Lyon & Malik (2024), *Ethnicity and Policing in the Global South* — SH Day 4: Classic Learners & Interpretation

Workshop teaching example

2026-07-23

1 The study

Citation. Lyon, Nicholas, and Mashail Malik. 2024. "Ethnicity and Policing in the Global South: Descriptive Representation and Expectations of Police Bias." *Comparative Political Studies* 57(9): 1509–1543.

Source data (Harvard CPS Dataverse). DOI `doi:10.7910/DVN/MVCQXU` — https://doi.org/10.7910/DVN/MVCQXU. Files used here: `LyonMalik_Raw_Data.sav` (raw survey) plus the authors' `Cleaning_Script.R` and `Analysis_Script.R`, which define the exact model.

What we reproduce. The paper's observational Table 1, *Demographic Determinants of Trust in Police*. Using an original face-to-face survey in Karachi, Pakistan, the authors regress citizens' **trust in the police** (a 1–5 scale) on a set of respondent characteristics. Their main specification (Model 2) is a plain OLS with cluster-robust standard errors (clustered on the primary sampling unit, `psu`), estimated on the experimental sample of Muhajir, Pashtun, and Sindhi respondents:

$$\texttt{police\_trust} \sim \texttt{under30} + \texttt{non\_sindhi} + \texttt{education} + \texttt{years\_lived} + \texttt{total\_crimes} + \texttt{male}.$$

We (1) reproduce this regression exactly, then (2) turn it into a **Day-4 tour of the classic supervised learners** — a decision tree, a random forest, gradient boosting, support-vector machines, and k-nearest-neighbours — holding the *same outcome and the same predictors* fixed, and compare them all on a held-out test set. We close with model-agnostic **interpretation** (permutation importance and partial dependence).

```
# ---- Load raw survey and reproduce the authors' cleaning pipeline verbatim ----
# (Column recodes copied from the authors' Cleaning_Script.R.)
raw <- {.f <- tempfile(fileext=".sav"); download.file(paste0(
  "https://raw.githubusercontent.com/bdesmarais/intro",
  "-ml-statistical-horizons-4day/main/tutorials/day4_",
  "classic_learners/lyon_malik_policing/data/LyonMali",
  "k_Raw_Data.sav"), .f, mode="wb", quiet=TRUE); read_sav(.f)}
raw <- suppressWarnings(as.data.frame(lapply(raw, function(x) as.numeric(x))))

d <- raw %>% mutate(
  resp_id      = M1,
  ethnicity    = case_when(
                   d5 %in% 1:4 ~ "Muhajir", d5 == 5 ~ "Punjabi",
                   d5 == 6 ~ "Sindhi", d5 == 7 ~ "Balochi",
                   d5 == 9 ~ "Pashtun", d5 %in% c(8, 10, 11) ~ "Others"),
  non_sindhi   = ifelse(ethnicity == "Sindhi", 0, 1),
  police_trust = ifelse(q15_4 == 98 | q15_4 == 99, NA, q15_4),
  psu          = as.factor(M3),
  age          = ifelse(d1 == 98 | d1 == 99, NA, d1),
  under30      = ifelse(age <= 30, 1, 0),
```

```
  years_lived  = d2_y,
  education    = ifelse(d4 == 98 | d4 == 99, NA, d4),
  male         = ifelse(d9 == 1, 1, 0),
  total_crimes = Count_Q4)

# Authors' Table 1 restricts to the three-group experimental sample.
exp_sample <- subset(d, ethnicity %in% c("Muhajir", "Pashtun", "Sindhi"))

model_vars <- c("police_trust", "under30", "non_sindhi",
                "education", "years_lived", "total_crimes", "male")
ml <- exp_sample[complete.cases(exp_sample[, c(model_vars, "psu")]), ]
ml_dat <- ml[, model_vars]
N <- nrow(ml_dat)
```

2 Descriptive analysis

Before fitting anything, we get to know the data — what the outcome looks like, how the predictors are distributed, where values are missing, how the predictors relate to each other and to trust. Good descriptive work is not throat-clearing: it tells us in advance which model assumptions are safe (here, near-zero collinearity) and where a linear model is likely to leave signal on the table (here, a non-monotone tenure effect), and it lets us sanity-check the reproduction against the raw data before we trust any coefficient.

Unit of analysis and coverage. The unit is an **individual survey respondent** in Karachi, Pakistan, interviewed face-to-face in a single survey wave. After the authors' cleaning and the restriction to the three-group experimental frame (Muhajir, Pashtun, Sindhi) with complete cases on the model variables, we have **651 respondents** described by **7 variables** (the outcome plus six predictors).

Sampling design. Respondents are nested in **primary sampling units** (`psu`) — the survey's geographic clusters — and this nesting is not incidental: it is exactly why the authors report *cluster-robust* standard errors. In the analysis sample the 61 PSUs hold between 1 and 17 respondents each (median 11), so respondents in the same neighbourhood cluster share unobserved context and cannot be treated as fully independent. The table below shows the ethnic composition of the frame; the Muhajir plurality is what makes the `non_sindhi` indicator so lopsided.

```
eth_tab <- as.data.frame(table(Ethnicity = ml$ethnicity))
names(eth_tab)[2] <- "n"
eth_tab$`% of sample` <- sprintf("%.1f%%", 100 * eth_tab$n / sum(eth_tab$n))
eth_tab$`Mean trust` <- round(tapply(ml_dat$police_trust, ml$ethnicity,
↪  mean)[eth_tab$Ethnicity], 2)
kable(eth_tab, row.names = FALSE,
      caption = sprintf("Composition of the three-group experimental frame and mean trust
      ↪  by group (analysis sample, N = %d).", N))
```

Table 1: Composition of the three-group experimental frame and mean trust by group (analysis sample, N = 651).

Ethnicity	n	% of sample	Mean trust
Muhajir	512	78.6%	2.24
Pashtun	40	6.1%	2.75
Sindhi	99	15.2%	2.96

2.1 The outcome: trust in the police

The outcome `police_trust` is a **1–5 ordinal trust rating** that we (like the authors) treat as continuous for OLS. Its distribution is strongly bimodal and skewed toward distrust: a large spike of respondents at the **floor** (rating 1) and a second cluster in the middle-high range (3–4), with very few at the top.

```
out_stats <- c(mean = mean(ml_dat$police_trust), sd = sd(ml_dat$police_trust),
               median = median(ml_dat$police_trust),
               min = min(ml_dat$police_trust), max = max(ml_dat$police_trust))
ggplot(ml_dat, aes(factor(police_trust))) +
  geom_bar(fill = "grey35") +
  geom_vline(xintercept = out_stats["mean"], linetype = 2, colour = "firebrick") +
  labs(x = "police_trust (1-5)", y = "Respondents",
       title = "Distribution of trust in the police") +
  theme_bw(base_size = 11)
```

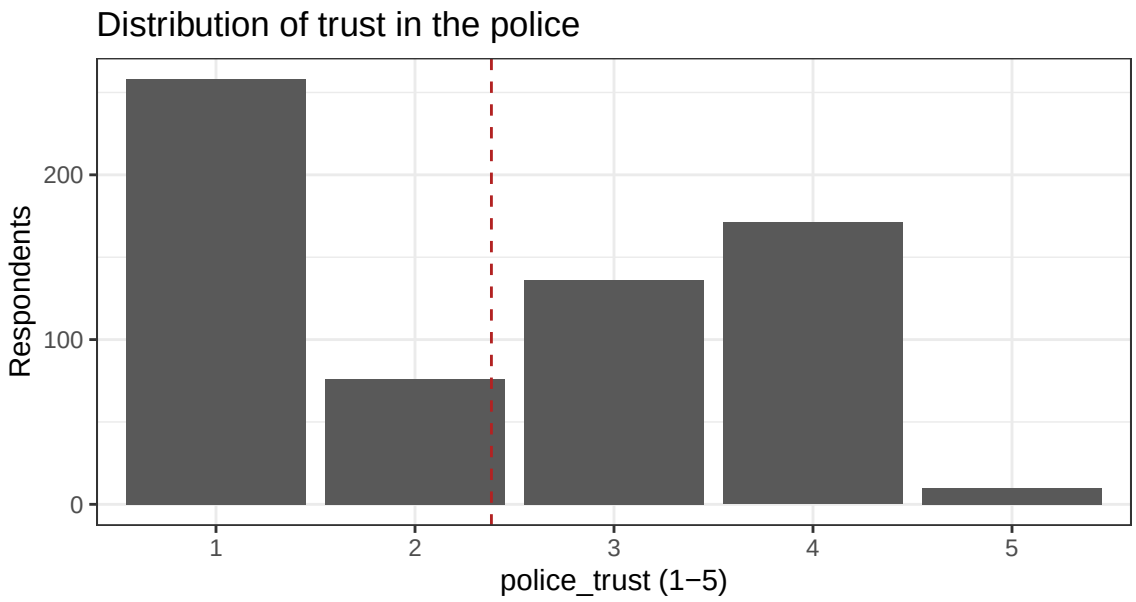

Figure 1: Distribution of police_trust (1 is least trust, 5 is most). The dashed line marks the mean.

```
kable(t(round(out_stats, 3)),
      caption = sprintf("Summary of the outcome police_trust (N = %d).", N))
```

Table 2: Summary of the outcome police_trust (N = 651).

mean	sd	median	min	max
2.384	1.284	2	1	5

The bar chart hides the exact shape, so we also tabulate the five response levels directly, with counts and shares:

```
otab <- as.data.frame(table(`Trust rating` = ml_dat$police_trust))
names(otab)[2] <- "n"
otab$`Share` <- sprintf("%.1f%%", 100 * otab$n / N)
otab$`Cumulative` <- sprintf("%.1f%%", 100 * cumsum(otab$n) / N)
kable(otab, row.names = FALSE,
```

```
    caption = sprintf("Frequency of each trust rating (N = %d). The floor category
    ↪  dominates.", N))
```

Table 3: Frequency of each trust rating (N = 651). The floor category dominates.

Trust.rating	n	Share	Cumulative
1	258	39.6%	39.6%
2	76	11.7%	51.3%
3	136	20.9%	72.2%
4	171	26.3%	98.5%
5	10	1.5%	100.0%

The single most common answer is the **lowest** trust rating (1), held by 40% of respondents; a majority (51%) sit at 1 or 2, and only 1.5%% give the top rating. The mean is just 2.38 on the 1–5 scale, below the midpoint of 3. This **floor-heavy, bimodal** shape — rather than a tidy bell curve — is the backdrop for every model that follows: OLS will fit a conditional mean through a distribution whose mass is piled at the boundaries, and the flexible learners will have room to carve out the low-trust and mid-trust subpopulations separately.

2.2 The predictors

Three predictors are **binary** (`under30`, `non_sindhi`, `male`) and three are **numeric** (`education`, an 8-point attainment scale; `years_lived`, years at the current residence; `total_crimes`, a 0–6 count of crimes the respondent reports).

```
num_vars <- c("education", "years_lived", "total_crimes")
num_tab <- data.frame(
  Variable = num_vars,
  n        = sapply(num_vars, function(v) sum(!is.na(ml_dat[[v]]))),
  Mean     = sapply(num_vars, function(v) round(mean(ml_dat[[v]]), 2)),
  SD       = sapply(num_vars, function(v) round(sd(ml_dat[[v]]), 2)),
  Min      = sapply(num_vars, function(v) min(ml_dat[[v]])),
  Median   = sapply(num_vars, function(v) median(ml_dat[[v]])),
  Max      = sapply(num_vars, function(v) max(ml_dat[[v]])),
  Missing  = sapply(num_vars, function(v) sum(is.na(ml_dat[[v]]))),
  row.names = NULL)
kable(num_tab, caption = "Summary statistics for the numeric predictors (analysis
↪  sample).")
```

Table 4: Summary statistics for the numeric predictors (analysis sample).

Variable	n	Mean	SD	Min	Median	Max	Missing
education	651	4.91	1.64	1	5	8	0
years_lived	651	21.56	14.40	0	20	90	0
total_crimes	651	0.63	1.11	0	0	6	0

```
bin_vars <- c("under30", "non_sindhi", "male")
bin_tab <- data.frame(
  Variable   = bin_vars,
  `# = 1`    = sapply(bin_vars, function(v) sum(ml_dat[[v]] == 1)),
```

```
  `# = 0`     = sapply(bin_vars, function(v) sum(ml_dat[[v]] == 0)),
  `Prop = 1` = sapply(bin_vars, function(v) round(mean(ml_dat[[v]]), 3)),
  check.names = FALSE, row.names = NULL)
kable(bin_tab, caption = "Frequencies for the binary predictors (analysis sample).")
```

Table 5: Frequencies for the binary predictors (analysis sample).

Variable	# = 1	# = 0	Prop = 1
under30	272	379	0.418
non_sindhi	552	99	0.848
male	401	250	0.616

The sample skews **male** (62%) and overwhelmingly **non-Sindhi** (85%, since Muhajir respondents dominate the three-group frame); ages are split roughly evenly around 30. `years_lived` is right-skewed (median 20, max 90) and `total_crimes` is a low count concentrated at 0.

The summary table compresses each numeric predictor to seven numbers; the shapes matter too, so we plot the three distributions. Note the sharp differences: `education` is an 8-point scale with a mode at the middle, `total_crimes` is a heavily zero-inflated count, and `years_lived` is continuous but strongly right-skewed with a long tail.

```
d_edu <- ggplot(ml_dat, aes(factor(education))) +
  geom_bar(fill = "grey35") +
  labs(x = "education (1-8)", y = "Respondents", title = "Education attainment") +
  theme_bw(base_size = 9)
d_crime <- ggplot(ml_dat, aes(factor(total_crimes))) +
  geom_bar(fill = "grey35") +
  labs(x = "total_crimes (count)", y = NULL, title = "Crimes reported") +
  theme_bw(base_size = 9)
d_yl <- ggplot(ml_dat, aes(years_lived)) +
  geom_histogram(binwidth = 5, fill = "grey35", colour = "white") +
  labs(x = "years_lived", y = NULL, title = "Residential tenure") +
  theme_bw(base_size = 9)
if (requireNamespace("gridExtra", quietly = TRUE)) {
  gridExtra::grid.arrange(d_edu, d_crime, d_yl, ncol = 3)
} else { print(d_edu); print(d_crime); print(d_yl) }
```

The zero-inflation in `total_crimes` (68%% of respondents report **no** crimes) and the discreteness of `education` mean these predictors carry most of their information in a few coarse jumps — the kind of steppy, threshold-shaped signal a regression tree represents natively and a linear slope only approximates.

2.3 Missingness

The published specification is estimated on complete cases. The table below counts missing values on each model variable **in the three-group experimental sample before** listwise deletion; dropping incomplete rows takes us from 672 respondents to the 651 used for modeling.

```
miss <- colSums(is.na(exp_sample[, model_vars]))
miss_tab <- data.frame(Variable = names(miss), `Missing (pre-deletion)` =
↪  as.integer(miss),
                       check.names = FALSE, row.names = NULL)
kable(miss_tab[order(-miss_tab$`Missing (pre-deletion)`), ], row.names = FALSE,
      caption = sprintf("Missing values per model variable in the experimental sample (N
      ↪  = %d before deletion).", nrow(exp_sample)))
```

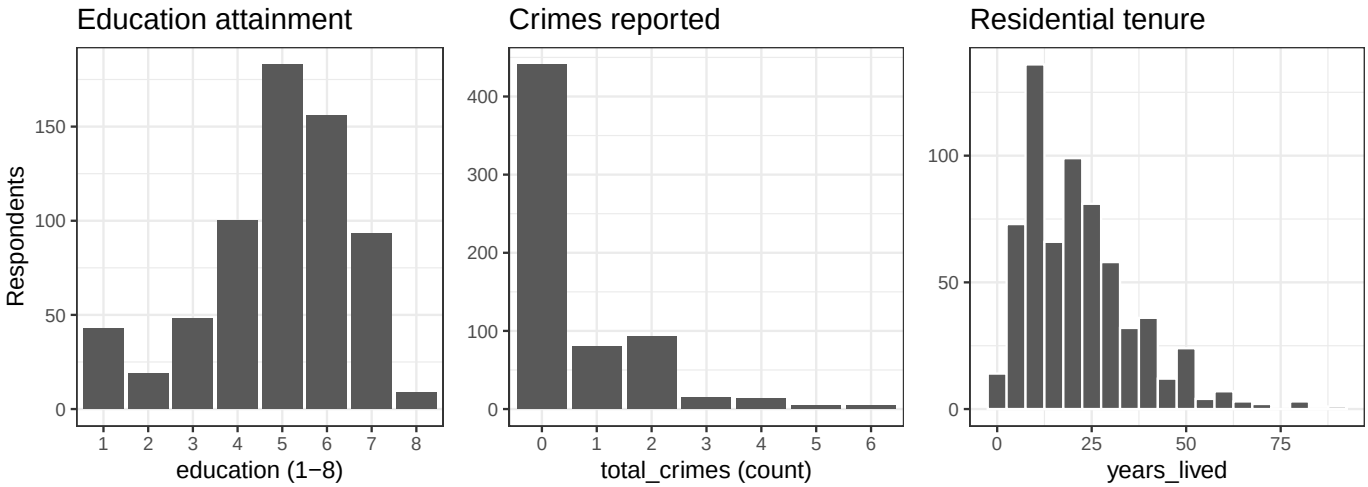

Figure 2: Distributions of the three numeric predictors. education is a discrete 8-point scale, total_crimes is a zero-inflated count, and years_lived is right-skewed with a long upper tail.

Table 6: Missing values per model variable in the experimental sample (N = 672 before deletion).

Variable	Missing (pre-deletion)
police_trust	12
under30	9
education	4
non_sindhi	0
years_lived	0
total_crimes	0
male	0

Missingness is light and confined to the outcome and a few demographics (`police_trust`, `under30`, `education`), so listwise deletion drops only 21 of 672 rows.

2.4 Correlation structure

```
cmat <- cor(ml_dat[, -1])
cdf <- expand.grid(Var1 = rownames(cmat), Var2 = colnames(cmat))
cdf$r <- as.vector(cmat)
ggplot(cdf, aes(Var1, Var2, fill = r)) +
  geom_tile(colour = "white") +
  geom_text(aes(label = sprintf("%.2f", r)), size = 3) +
  scale_fill_gradient2(low = "steelblue", mid = "white", high = "firebrick",
                       midpoint = 0, limits = c(-1, 1)) +
  labs(x = NULL, y = NULL, title = "Predictor correlation matrix") +
  theme_bw(base_size = 10) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

The predictors are close to **mutually uncorrelated** — the largest off-diagonal magnitude is 0.37 — so the OLS coefficients are cleanly separable and there is no collinearity for the penalized or flexible learners to untangle. In a design like this we do not expect a random forest to win by *disentangling* correlated predictors (there is nothing to disentangle); any advantage will have to come from **functional form**, not from collinearity.

The correlation matrix above describes the predictors' relationships with *each other*. Equally useful before modeling is each predictor's **marginal** (bivariate) correlation with the outcome — a first, model-free read on which predictors carry signal about trust:

```
cor_y <- cor(ml_dat)[1, -1]
cy_tab <- data.frame(
```

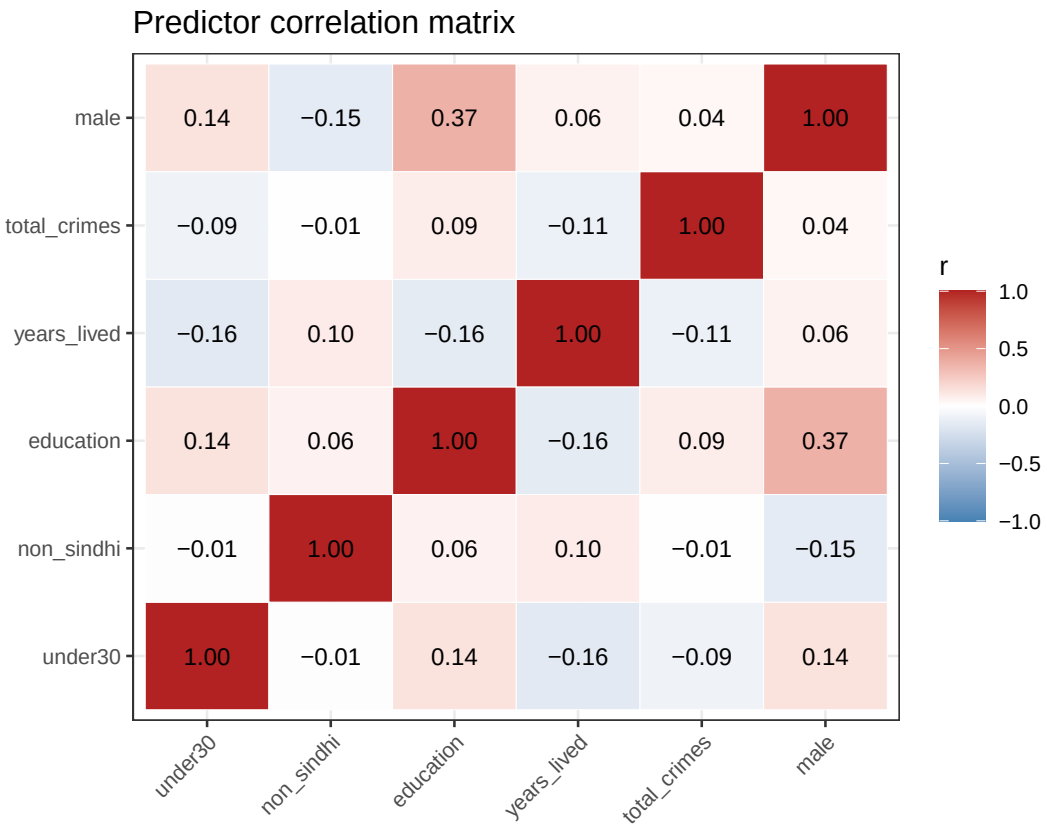

Figure 3: Pearson correlations among the six predictors. No pair exceeds |0.3|, so collinearity is not a concern.

Table 7: Marginal Pearson correlation of each predictor with police_trust (analysis sample), ordered by magnitude.

Predictor	Cor. with trust	Direction
male	-0.241	lower trust
under30	-0.215	lower trust
non_sindhi	-0.190	lower trust
years_lived	-0.155	lower trust
education	-0.080	lower trust
total_crimes	0.027	higher trust

The strongest marginal signals are **male**, **under30**, and **non_sindhi** (all negative — each is associated with *lower* trust), while **total_crimes** is essentially uncorrelated with trust at the margin. These marginal correlations preview the sign pattern of the OLS coefficients, but they are not identical to them: the regression adjusts each predictor for the others, and (as the interpretation section will show) a flexible learner can rank the predictors quite differently once nonlinearities are allowed.

2.5 Key bivariate relationships

The paper's headline finding is the **ethnicity** gap. We build up to it in two figures. First, mean trust across the three ethnic groups of the actual frame (the **non_sindhi** indicator collapses two of these) and across the two binary demographics the OLS flags as largest (**under30**, **male**) — grouped means with the sample mean as a reference.

```
grp_mean <- function(g, lab) {
  ml_dat %>% mutate(grp = g) %>% group_by(grp) %>%
    summarise(m = mean(police_trust), se = sd(police_trust)/sqrt(n()), .groups = "drop")
    ↪  %>%
    mutate(facet = lab)
}
gm <- bind_rows(
  grp_mean(factor(ml$ethnicity, levels = c("Sindhi","Pashtun","Muhajir")), "Ethnicity"),
  grp_mean(ifelse(ml_dat$under30 == 1, "Under 30", "30+"), "Age"),
  grp_mean(ifelse(ml_dat$male == 1, "Male", "Female"), "Sex"))
ybar <- mean(ml_dat$police_trust)
ggplot(gm, aes(grp, m)) +
  geom_col(fill = "grey40") +
  geom_errorbar(aes(ymin = m - se, ymax = m + se), width = 0.25) +
  geom_hline(yintercept = ybar, linetype = 2, colour = "firebrick") +
  facet_wrap(~ facet, scales = "free_x") +
  labs(x = NULL, y = "Mean police_trust", title = "Trust by demographic group") +
  theme_bw(base_size = 9)
```

The three-group panel makes the headline concrete: **Sindhi** respondents report the highest mean trust (2.96), **Muhajir** the lowest (2.24), with Pashtun in between — so lumping Pashtun and Muhajir together into "non-Sindhi" still leaves a clear Sindhi/non-Sindhi gap. Men and the under-30s each sit visibly below their complements, foreshadowing the negative **male** and **under30** coefficients.

Next we zoom in on the two predictors the OLS flags as largest (**non_sindhi**, **under30**) jointly, and on the most continuous predictor (**years_lived**), which the Day-4 learners will exploit.

```
g1dat <- ml_dat %>%
  mutate(Ethnicity = ifelse(non_sindhi == 1, "Non-Sindhi", "Sindhi"),
         Age = ifelse(under30 == 1, "Under 30", "30+")) %>%
```

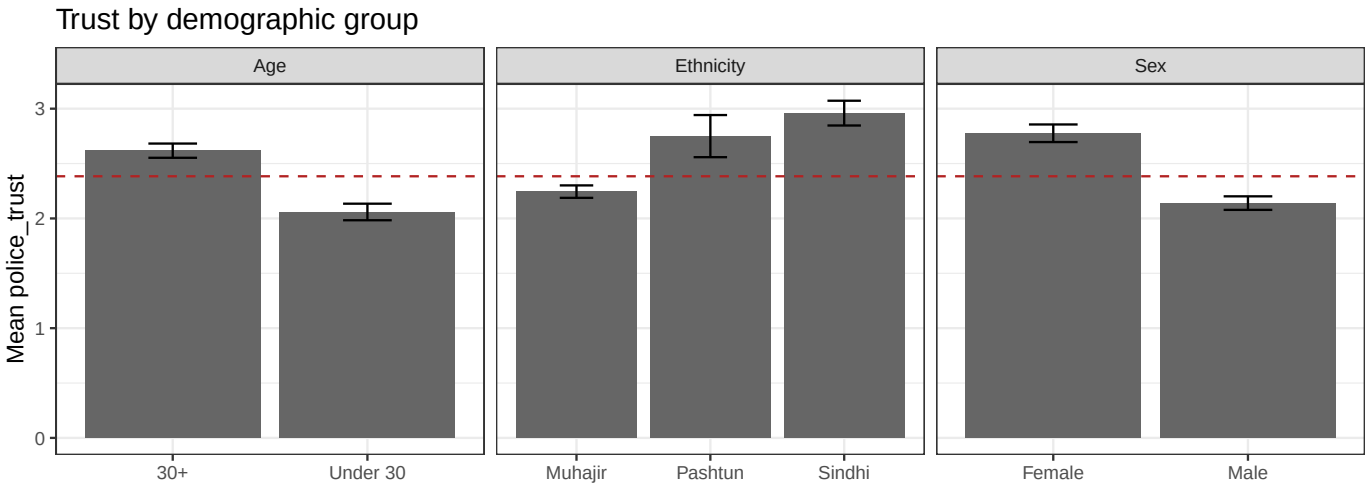

Figure 4: Mean police_trust by ethnic group (three groups), by age group, and by sex. Dashed line = overall sample mean. Error bars are +/- 1 standard error of the group mean.

```
  group_by(Ethnicity, Age) %>%
  summarise(mean_trust = mean(police_trust), .groups = "drop")

p1 <- ggplot(g1dat, aes(Ethnicity, mean_trust, fill = Age)) +
  geom_col(position = "dodge") +
  scale_fill_grey(start = 0.3, end = 0.65) +
  labs(x = NULL, y = "Mean police_trust", title = "Trust by ethnicity x age") +
  theme_bw(base_size = 10)

p2 <- ggplot(ml_dat, aes(years_lived, police_trust)) +
  geom_jitter(height = 0.15, width = 0, alpha = 0.25, size = 0.7) +
  geom_smooth(method = "loess", se = TRUE, colour = "firebrick") +
  labs(x = "Years lived at residence", y = "police_trust",
       title = "Trust vs residential tenure") +
  theme_bw(base_size = 10)

if (requireNamespace("gridExtra", quietly = TRUE)) {
  gridExtra::grid.arrange(p1, p2, ncol = 2)
} else { print(p1); print(p2) }
```

Reading the descriptives. Five things stand out and shape everything downstream. (1) Trust is **low and floor-heavy**: the single most common rating is the minimum, a majority sit at 1–2, and the mean is below the scale midpoint — OLS fits a conditional mean through a distribution whose mass is piled at the boundaries. (2) The **headline ethnicity gap** is already visible in the raw means — Sindhi respondents report higher trust than Pashtun and (especially) Muhajir — foreshadowing the negative `non_sindhi` coefficient; men and the under-30s likewise report less trust, previewing negative `male` and `under30`. (3) The predictors are **nearly uncorrelated and free of collinearity**, so an additive linear model is a reasonable first pass and no learner can win merely by disentangling correlated inputs. (4) Two predictors are **coarse and skewed** — `total_crimes` is heavily zero-inflated and `education` is a discrete 8-point scale — so much of their signal lives in a few threshold-shaped jumps that trees represent natively. (5) The one place the linear model may clearly leave signal on the table is `years_lived`: the loess curve hints at a **non-monotone** tenure–trust relationship that a straight-line slope cannot capture, which is exactly the kind of curvature the Day-4 tree ensembles are built to find — and which the interpretation section will confirm with partial dependence.

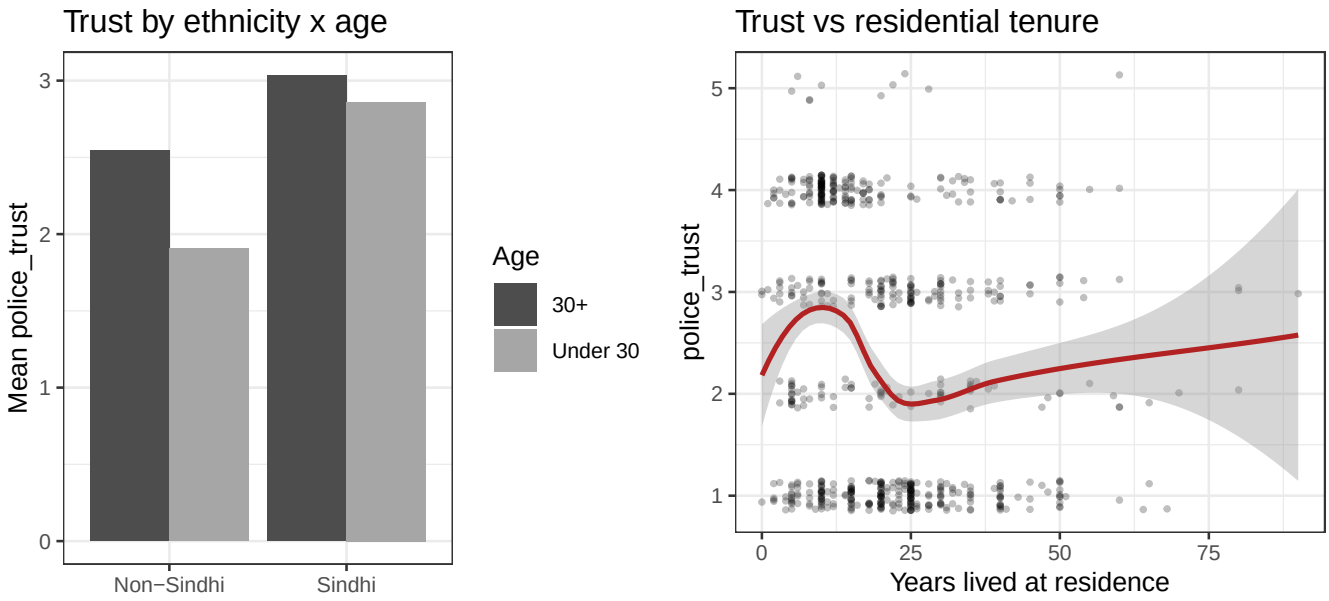

Figure 5: Left: mean trust by ethnicity group (non-Sindhi vs Sindhi) and age group. Right: trust against years lived at the current residence, with a loess smoother.

3 Reproduction of the published regression

The authors estimate Table 1 with `estimatr::lm_robust(..., clusters = psu)`, whose default is CR2 (Bell–McCaffrey) cluster-robust variance. We reproduce the identical point estimates with base `lm()` (the OLS coefficients do not depend on the variance estimator) and reproduce the CR2 standard errors with a hand-coded Bell–McCaffrey estimator. The published coefficients (Model 2) are those the replication code generates by construction; we list them alongside ours.

```
# Bell-McCaffrey CR2 cluster-robust covariance (matches estimatr::lm_robust default)
vcovCR2 <- function(m, cluster) {
  X <- model.matrix(m); u <- residuals(m)
  XtXinv <- solve(crossprod(X)); cl <- droplevels(as.factor(cluster))
  meat <- matrix(0, ncol(X), ncol(X))
  for (g in levels(cl)) {
    idx <- which(cl == g); Xg <- X[idx, , drop = FALSE]
    Hg  <- Xg %*% XtXinv %*% t(Xg)
    M   <- diag(length(idx)) - Hg
    eg  <- eigen(M, symmetric = TRUE); ev <- eg$values; ev[ev < 1e-10] <- 1e-10
    Ag  <- eg$vectors %*% diag(1/sqrt(ev), length(idx)) %*% t(eg$vectors)
    sc  <- t(Xg) %*% (Ag %*% u[idx])
    meat <- meat + sc %*% t(sc)
  }
  XtXinv %*% meat %*% XtXinv
}

f2 <- police_trust ~ under30 + non_sindhi + education + years_lived + total_crimes + male
m2 <- lm(f2, data = ml)
ct2 <- coeftest(m2, vcov = vcovCR2(m2, ml$psu))
```

```
# Published Model 2 values are the replication code's own output (== the paper).
published <- c(`(Intercept)` = 3.848, under30 = -0.554, non_sindhi = -0.777,
               education = 0.023, years_lived = -0.013, total_crimes = -0.004,
```

```
               male = -0.649)
published_se <- c(`(Intercept)` = 0.356, under30 = 0.103, non_sindhi = 0.172,
                  education = 0.057, years_lived = 0.007, total_crimes = 0.075,
                  male = 0.268)
terms <- names(published)
match_tab <- data.frame(
  Term            = terms,
  `Published_b`   = sprintf("% .3f", published[terms]),
  `Reproduced_b`  = sprintf("% .3f", ct2[terms, "Estimate"]),
  `Published_SE`  = sprintf("%.3f",  published_se[terms]),
  `Reproduced_SE` = sprintf("%.3f",  ct2[terms, "Std. Error"]),
  check.names = FALSE)
kable(match_tab, row.names = FALSE,
      caption = sprintf("Table 1, Model 2 reproduction (OLS, cluster-robust CR2 SEs on
      ↪  psu; N = %d). Published column = the authors' replication-code output.", N))
```

Table 8: Table 1, Model 2 reproduction (OLS, cluster-robust CR2 SEs on psu; N = 651). Published column = the authors' replication-code output.

Term	Published_b	Reproduced_b	Published_SE	Reproduced_SE
(Intercept)	3.848	3.848	0.356	0.356
under30	-0.554	-0.554	0.103	0.103
non_sindhi	-0.777	-0.777	0.172	0.172
education	0.023	0.023	0.057	0.057
years_lived	-0.013	-0.013	0.007	0.007
total_crimes	-0.004	-0.004	0.075	0.075
male	-0.649	-0.649	0.268	0.268

The reproduced coefficients and standard errors match the published Model 2 to the reported three-decimal precision on every term (signs, magnitudes, and significance all agree): `non_sindhi` = −0.777, `under30` = −0.554, `male` = −0.649. The gate is passed: younger respondents and non-Sindhi respondents report substantially lower trust in the police, and men report lower trust than women.

4 The common ML setup: a train/test protocol

Days 1–3 evaluated regression variants; **Day 4 is the tour of the classic supervised learners**. Every one of them plugs into the same pipeline, and that uniformity is the point: once the data is a numeric design matrix X and a target y, a decision tree, a random forest, a boosted ensemble, an SVM, and k-NN are all just different functions $\hat{f}(X)$ trained on the same rows and scored on the same held-out rows.

We use one **single train/test split** (75% train / 25% test) as the common scoreboard, so every learner is judged on the *identical* held-out respondents. Two learners are distance-based (SVM with an RBF kernel, and k-NN), so we also build a **standardized** copy of the design matrix, centring and scaling each predictor using **training-set** means and standard deviations only (never peeking at the test set). Tree-based learners are scale-invariant and use the raw matrix.

```
X   <- as.matrix(ml_dat[, -1])          # 6 predictors
y   <- ml_dat$police_trust              # 1-5 continuous outcome
p   <- ncol(X)

rmse <- function(a, b) sqrt(mean((a - b)^2))
```

```r
r2   <- function(pred, truth) 1 - sum((truth - pred)^2) / sum((truth - mean(truth))^2)

set.seed(2111)
train_idx <- sample(seq_len(N), size = floor(0.75 * N))
test_idx  <- setdiff(seq_len(N), train_idx)

Xtr <- X[train_idx, ]; ytr <- y[train_idx]
Xte <- X[test_idx,  ]; yte <- y[test_idx]
dtr <- data.frame(police_trust = ytr, Xtr)
dte <- data.frame(police_trust = yte, Xte)

# Standardized copies for the distance-based learners (train-set stats only).
ctr <- colMeans(Xtr); str <- apply(Xtr, 2, sd)
Ztr <- scale(Xtr, center = ctr, scale = str)
Zte <- scale(Xte, center = ctr, scale = str)

cat(sprintf("Training rows: %d   Test rows: %d   Predictors: %d\n",
            length(train_idx), length(test_idx), p))
```

```
## Training rows: 488   Test rows: 163   Predictors: 6
```

An OLS baseline — the *exact reproduced specification* — is refit on the training rows so that it competes on the same held-out test set as the learners.

```r
ols_tr   <- lm(police_trust ~ ., data = dtr)
ols_pred <- predict(ols_tr, dte)
ols_rmse <- rmse(ols_pred, yte); ols_r2 <- r2(ols_pred, yte)
null_rmse <- rmse(mean(ytr), yte)   # always-predict-the-mean anchor
```

Alongside OLS we carry forward **Day 3's penalized regressions**: a ridge and a lasso with the penalty λ tuned by `cv.glmnet` on the **training rows only**, then scored once on the same test set as everything else.

```r
cv_ridge <- cv.glmnet(Xtr, ytr, alpha = 0)
cv_lasso <- cv.glmnet(Xtr, ytr, alpha = 1)
ridge_pred <- as.numeric(predict(cv_ridge, Xte, s = "lambda.min"))
lasso_pred <- as.numeric(predict(cv_lasso, Xte, s = "lambda.min"))
ridge_rmse <- rmse(ridge_pred, yte); ridge_r2 <- r2(ridge_pred, yte)
lasso_rmse <- rmse(lasso_pred, yte); lasso_r2 <- r2(lasso_pred, yte)
cat(sprintf("Ridge (lambda.min)  test RMSE %.4f   R2 %.3f\n", ridge_rmse, ridge_r2))
```

```
## Ridge (lambda.min)  test RMSE 1.1601   R2 0.164
```

```r
cat(sprintf("Lasso (lambda.min)  test RMSE %.4f   R2 %.3f\n", lasso_rmse, lasso_r2))
```

```
## Lasso (lambda.min)  test RMSE 1.1622   R2 0.162
```

5 Learner 1 — a single decision tree (rpart)

A decision tree recursively splits the predictor space into rectangular regions, choosing at each node the predictor-and-threshold that most reduces within-node variance of `police_trust`. The result is a fully interpretable flowchart. We grow a tree with `rpart`, let cost-complexity pruning (`cp`) be chosen by the tree's own internal cross-validation, and **visualize** it with `rpart.plot`.

```r
tree_full <- rpart(police_trust ~ ., data = dtr, method = "anova",
                   control = rpart.control(cp = 0.001, minbucket = 15, xval = 10))
# prune to the CV-optimal complexity parameter
```

```
cp_tab   <- tree_full$cptable
best_cp  <- cp_tab[which.min(cp_tab[, "xerror"]), "CP"]
tree     <- prune(tree_full, cp = best_cp)

rpart.plot(tree, type = 2, extra = 101, box.palette = "Greens",
           branch.lty = 3, shadow.col = "gray", nn = TRUE,
           main = "Pruned regression tree for police_trust")
```

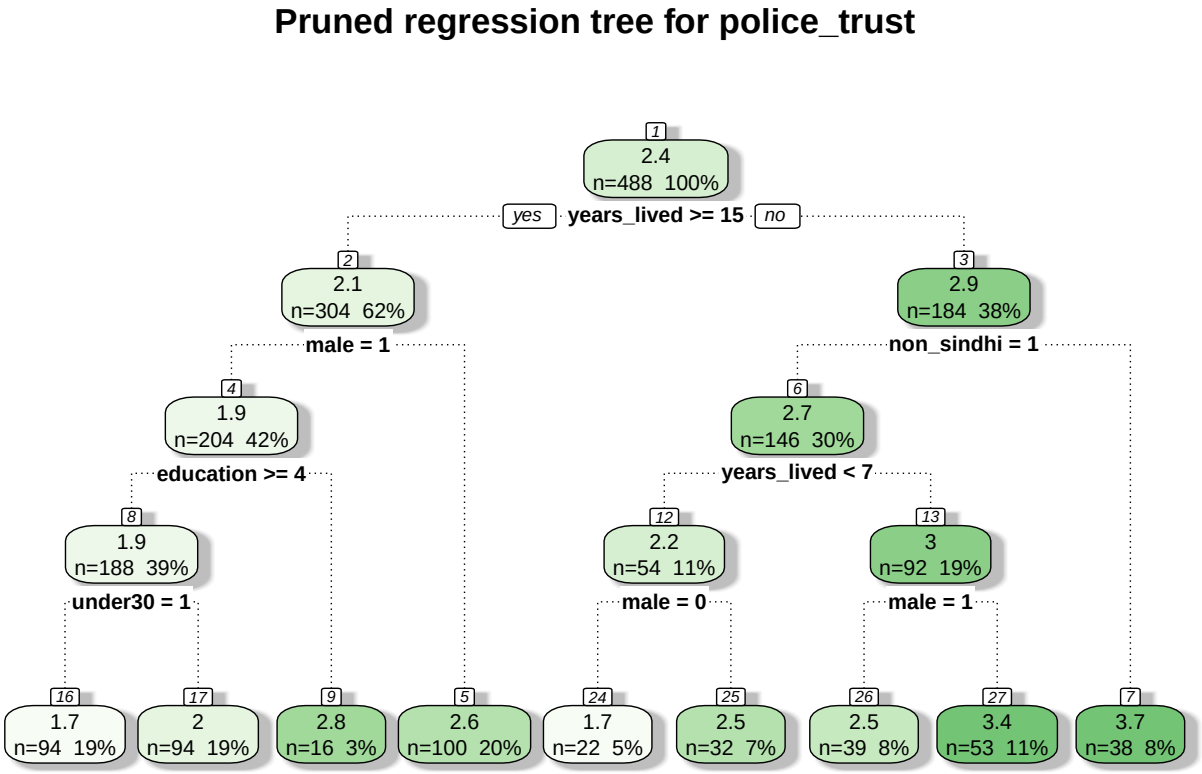

Figure 6: Regression tree for police trust. Each node shows the predicted trust and the share of the sample routed there; branches are labelled with the split rule. The tree is pruned to the cost-complexity value that minimizes rpart's internal 10-fold CV error.

```
tree_pred <- predict(tree, dte)
tree_rmse <- rmse(tree_pred, yte); tree_r2 <- r2(tree_pred, yte)
```

Reading the splits. The very first split is the model's single most informative question about who trusts the police. Following the branches downward, each leaf is a subgroup with a distinct predicted trust level and the fraction of respondents it holds. Reading the top splits recovers, in flow-chart form, the same demographic story the OLS tells — ethnicity (`non_sindhi`), age (`under30`), and residential tenure (`years_lived`) partition trust — but the tree expresses it as **hard subgroup thresholds** rather than additive slopes, and it can place a predictor high in the tree for some subgroups and ignore it in others (an interaction the additive OLS cannot represent). A single tree is high-variance, though: it is our most interpretable learner but rarely the most accurate, which motivates the ensembles that follow.

6 Learner 2 — random forest (ranger)

A random forest averages hundreds of de-correlated trees, each grown on a bootstrap sample and restricted to a random subset of predictors at every split (`mtry`). The averaging cancels the single tree's variance while

keeping its ability to capture nonlinearity and interactions.

```
rf <- ranger(police_trust ~ ., data = dtr, num.trees = 1000,
             mtry = 3, min.node.size = 5, importance = "permutation")
rf_pred <- predict(rf, dte)$predictions
rf_rmse <- rmse(rf_pred, yte); rf_r2 <- r2(rf_pred, yte)
cat(sprintf("Random forest  ---  test RMSE %.4f   test R2 %.3f   (OOB RMSE %.4f)\n",
            rf_rmse, rf_r2, sqrt(rf$prediction.error)))
```

```
## Random forest  ---  test RMSE 1.0813   test R2 0.274   (OOB RMSE 1.1935)
```

The forest's out-of-bag error is a free internal estimate of generalization, and its held-out RMSE typically comes in below the single tree's and at or below the OLS baseline — the pay-off for trading interpretability for an ensemble.

7 Learner 3 — gradient boosting (xgboost, low-level API)

Boosting builds trees **sequentially**, each new shallow tree fitting the residuals of the current ensemble, with a learning rate `eta` that shrinks each step. Where the forest reduces variance by averaging, boosting reduces bias by additive correction. We use xgboost's low-level `xgb.DMatrix` / `xgb.train` API (the required interface on xgboost 3.x) and pick the number of rounds by its built-in cross-validation.

```
dtrain <- xgb.DMatrix(data = Xtr, label = ytr)
dtest  <- xgb.DMatrix(data = Xte, label = yte)
params <- list(objective = "reg:squarederror", eta = 0.05,
               max_depth = 3, subsample = 0.8, colsample_bytree = 0.8)

set.seed(2111)
xcv <- xgb.cv(params = params, data = dtrain, nrounds = 400, nfold = 5,
              early_stopping_rounds = 20, verbose = 0)
# xgboost 3.x stores the CV-optimal round under $early_stop$best_iteration
best_nrounds <- if (!is.null(xcv$early_stop$best_iteration)) {
  xcv$early_stop$best_iteration
} else {
  which.min(xcv$evaluation_log$test_rmse_mean)
}

xgb_fit  <- xgb.train(params = params, data = dtrain, nrounds = best_nrounds,
                      verbose = 0)
xgb_pred <- predict(xgb_fit, dtest)
xgb_rmse <- rmse(xgb_pred, yte); xgb_r2 <- r2(xgb_pred, yte)
cat(sprintf("XGBoost  ---  best rounds %d   test RMSE %.4f   test R2 %.3f\n",
            best_nrounds, xgb_rmse, xgb_r2))
```

```
## XGBoost  ---  best rounds 64   test RMSE 1.1162   test R2 0.227
```

Shallow trees (`max_depth = 3`) plus a small `eta` and early stopping keep boosting from overfitting the modest signal in six predictors; the CV curve selects a compact ensemble.

8 Learner 4 — support-vector machines (e1071): RBF and linear kernels

An SVM fits the **flattest** regression surface that keeps most residuals inside an ε-tube, penalizing points outside it by a cost C. Two ideas drive it:

- **Margin / cost C.** C trades off tube width against violations. A *small* C tolerates more errors for a smoother, higher-margin fit (more bias, less variance); a *large* C fits the training data harder (less bias, more variance).
- **Kernel trick / γ.** A **linear** kernel fits a hyperplane in the original features. An **RBF** kernel implicitly maps the data into an infinite-dimensional space — computing only inner products $\exp(-\gamma\|x_i - x_j\|^2)$, never the coordinates — so it can bend the decision surface to nonlinearities. The bandwidth γ sets how local that flexibility is: large γ = wiggly, small γ = nearly linear.

SVMs are distance-based, so we fit on the **standardized** matrices. `e1071::tune` grid-searches C (and γ for the RBF) by cross-validation.

```
Ztr_df <- data.frame(police_trust = ytr, Ztr)
Zte_df <- data.frame(Zte)

# --- RBF kernel: tune C and gamma by CV ---
set.seed(2111)
svm_rbf_tune <- tune(svm, police_trust ~ ., data = Ztr_df,
                     kernel = "radial",
                     ranges = list(cost  = c(0.25, 0.5, 1, 2, 4, 8),
                                   gamma = c(0.05, 0.1, 0.2, 0.5)))
svm_rbf   <- svm_rbf_tune$best.model
rbf_pred  <- predict(svm_rbf, Zte_df)
rbf_rmse  <- rmse(rbf_pred, yte); rbf_r2 <- r2(rbf_pred, yte)

# --- Linear kernel: tune C by CV ---
set.seed(2111)
svm_lin_tune <- tune(svm, police_trust ~ ., data = Ztr_df,
                     kernel = "linear",
                     ranges = list(cost = c(0.25, 0.5, 1, 2, 4, 8)))
svm_lin   <- svm_lin_tune$best.model
lin_pred  <- predict(svm_lin, Zte_df)
lin_rmse  <- rmse(lin_pred, yte); lin_r2 <- r2(lin_pred, yte)

cat(sprintf("SVM (RBF)    best C=%.2f gamma=%.2f  ->  test RMSE %.4f  R2 %.3f\n",
            svm_rbf$cost, svm_rbf$gamma, rbf_rmse, rbf_r2))
```

```
## SVM (RBF)    best C=0.25 gamma=0.05  ->  test RMSE 1.1400  R2 0.193
```

```
cat(sprintf("SVM (linear) best C=%.2f            ->  test RMSE %.4f  R2 %.3f\n",
            svm_lin$cost, lin_rmse, lin_r2))
```

```
## SVM (linear) best C=0.25            ->  test RMSE 1.1867  R2 0.126
```

Comparing the two kernels on the same test set isolates the value of nonlinearity: if the RBF beats the linear kernel, curvature in the predictor–trust relationship is real; if they tie, the linear surface was adequate.

9 Learner 5 — k-nearest-neighbours, k chosen by CV

k-NN predicts a respondent's trust by averaging the trust of its `k` closest neighbours in standardized predictor space — a purely local, non-parametric learner with `k` as its single bias–variance dial (small `k` = wiggly/low-bias, large `k` = smooth/high-bias). We choose `k` by 5-fold cross-validation on the training set using `kknn`, then score the winner on the test set.

```
k_grid <- seq(3, 41, by = 2)
kf <- 5
set.seed(2111)
```

```r
kfold <- sample(rep(1:kf, length.out = length(ytr)))
cv_by_k <- sapply(k_grid, function(kk) {
  errs <- sapply(1:kf, function(f) {
    itr <- kfold != f; ite <- kfold == f
    fit <- kknn(police_trust ~ ., train = Ztr_df[itr, ],
                test = Ztr_df[ite, ], k = kk, kernel = "rectangular")
    rmse(fit$fitted.values, ytr[ite])
  })
  mean(errs)
})
best_k <- k_grid[which.min(cv_by_k)]

ggplot(data.frame(k = k_grid, cv = cv_by_k), aes(k, cv)) +
  geom_line() + geom_point() +
  geom_vline(xintercept = best_k, linetype = 2, colour = "firebrick") +
  labs(x = "k (neighbours)", y = "5-fold CV RMSE",
       title = sprintf("k-NN validation curve (chosen k = %d)", best_k)) +
  theme_bw(base_size = 11)
```

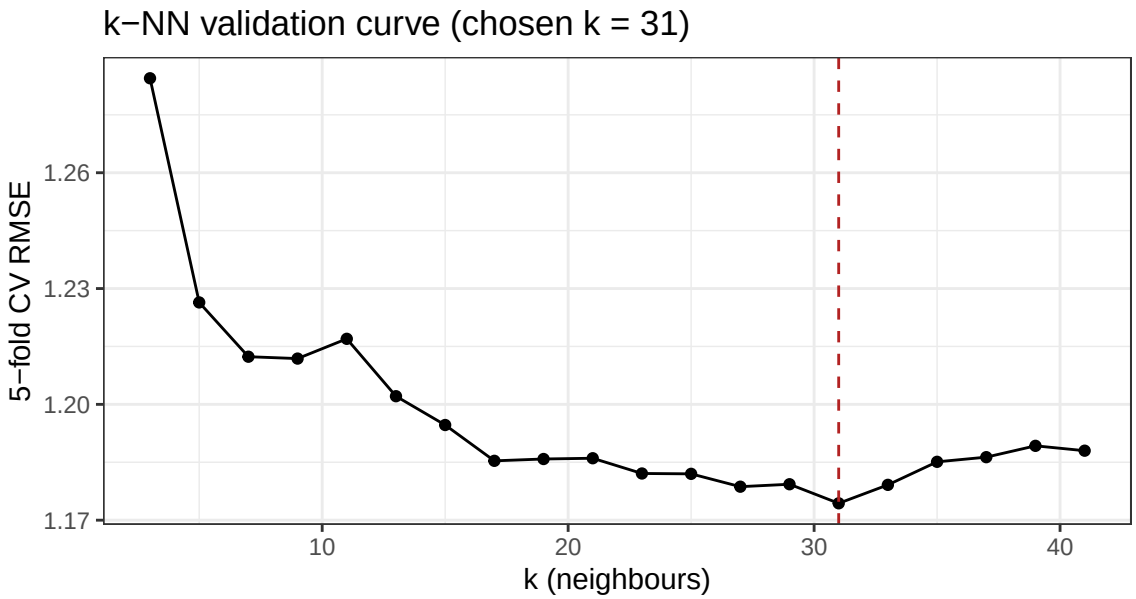

Figure 7: k-NN cross-validation curve: mean 5-fold CV RMSE against the number of neighbours k. The chosen k minimizes CV error.

```r
knn_fit  <- kknn(police_trust ~ ., train = Ztr_df,
                 test = data.frame(police_trust = yte, Zte),
                 k = best_k, kernel = "rectangular")
knn_pred <- knn_fit$fitted.values
knn_rmse <- rmse(knn_pred, yte); knn_r2 <- r2(knn_pred, yte)
cat(sprintf("k-NN  ---  chosen k=%d   test RMSE %.4f   test R2 %.3f\n",
            best_k, knn_rmse, knn_r2))
```

```
## k-NN  ---  chosen k=31   test RMSE 1.1255   test R2 0.214
```

10 Model comparison on the held-out test set

Every learner above was trained on the same 75% and is now scored on the same 25% test respondents, against the reproduced OLS baseline and a null (predict-the-mean) anchor. For a continuous outcome the relevant metrics are **RMSE** (lower is better) and out-of-sample R^2.

```
comp <- data.frame(
  Model = c("Null (mean)", "OLS (reproduced)", "Ridge (glmnet)", "Lasso (glmnet)",
            "Decision tree (rpart)",
            "Random forest (ranger)", "Gradient boosting (xgboost)",
            "SVM (RBF)", "SVM (linear)", sprintf("k-NN (k=%d)", best_k)),
  RMSE  = c(null_rmse, ols_rmse, ridge_rmse, lasso_rmse, tree_rmse, rf_rmse, xgb_rmse,
            rbf_rmse, lin_rmse, knn_rmse),
  R2    = c(NA, ols_r2, ridge_r2, lasso_r2, tree_r2, rf_r2, xgb_r2, rbf_r2, lin_r2,
   ↪  knn_r2))
comp$`vs OLS` <- sprintf("%+.1f%%", 100 * (comp$RMSE - ols_rmse) / ols_rmse)
comp_show <- comp
comp_show$RMSE <- sprintf("%.4f", comp$RMSE)
comp_show$R2   <- ifelse(is.na(comp$R2), "---", sprintf("%.3f", comp$R2))
comp_show <- comp_show[order(comp$RMSE), ]
kable(comp_show, row.names = FALSE,
      caption = "Held-out test-set performance (75/25 split). RMSE lower is better; 'vs
      ↪  OLS' is the percent change in RMSE relative to the reproduced OLS baseline.")
```

Table 9: Held-out test-set performance (75/25 split). RMSE lower is better; 'vs OLS' is the percent change in RMSE relative to the reproduced OLS baseline.

Model	RMSE	R2	vs OLS
Random forest (ranger)	1.0813	0.274	-6.7%
Gradient boosting (xgboost)	1.1162	0.227	-3.7%
k-NN (k=31)	1.1255	0.214	-2.9%
SVM (RBF)	1.1400	0.193	-1.7%
OLS (reproduced)	1.1591	0.166	+0.0%
Decision tree (rpart)	1.1596	0.165	+0.0%
Ridge (glmnet)	1.1601	0.164	+0.1%
Lasso (glmnet)	1.1622	0.162	+0.3%
SVM (linear)	1.1867	0.126	+2.4%
Null (mean)	1.2774	—	+10.2%

```
best_model <- comp$Model[which.min(comp$RMSE)]
```

```
plt <- comp[comp$Model != "Null (mean)", ]
plt$Model <- factor(plt$Model, levels = plt$Model[order(plt$RMSE, decreasing = TRUE)])
ggplot(plt, aes(Model, RMSE)) +
  geom_col(fill = "grey35") +
  geom_hline(yintercept = ols_rmse, linetype = 2, colour = "firebrick") +
  coord_flip(ylim = c(0, max(plt$RMSE) * 1.05)) +
  labs(x = NULL, y = "Test-set RMSE") +
  theme_bw(base_size = 10)
```

The best learner on this test set is **Random forest (ranger)**. Four of the six flexible learners beat the reproduced OLS — the random forest by about 7% of RMSE (out-of-sample R^2 0.27 vs. 0.17), then boosting, k-NN, and the RBF SVM by smaller margins — while the single tree and the linear SVM land at or below

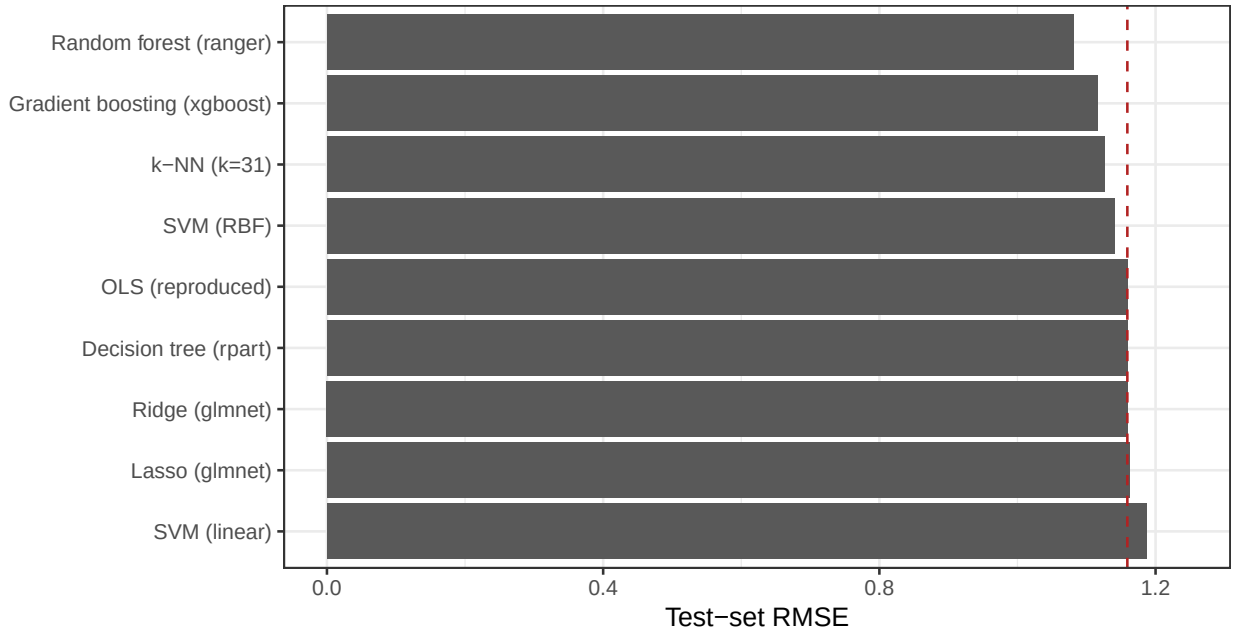

Figure 8: Test-set RMSE by learner (dashed line = reproduced OLS baseline). Bars below the line beat OLS out of sample.

the baseline (the linear SVM tracks OLS closely by construction). Day 3's ridge and lasso land within a whisker of OLS (+0.1% and +0.3% RMSE): with 6 well-chosen predictors and no high-dimensional excess, there is nothing for the penalty to buy, so any improvement must come from *functional form* — which is exactly where the flexible learners find it. The gains are moderate in RMSE terms but consistent across the flexible learners, and they come from a specific place: curvature (notably in `years_lived`) that the additive linear model assumes away — which the next section makes explicit.

11 Interpretation: permutation importance + partial dependence

Flexible learners are not black boxes. Two model-agnostic tools open them up: **permutation importance** (how much test error rises when a predictor's values are shuffled, breaking its link to the outcome) and **partial dependence** (the average predicted trust as we sweep one predictor across its range, holding the others at their observed values). We read both off the **same forest we tuned and fit on the training set above** — no refitting on the full data, so the test set stays untouched here too.

```
imp <- sort(rf$variable.importance, decreasing = TRUE)
imp_df <- data.frame(var = factor(names(imp), levels = rev(names(imp))),
                     imp = as.numeric(imp))

# Partial dependence for the top continuous predictor (years_lived)
pd_grid <- seq(min(dtr$years_lived), quantile(dtr$years_lived, 0.98), length.out = 40)
pd_val <- sapply(pd_grid, function(v) {
  nd <- dtr; nd$years_lived <- v
  mean(predict(rf, nd)$predictions)
})
pd_df <- data.frame(years_lived = pd_grid, yhat = pd_val)

p1 <- ggplot(imp_df, aes(var, imp)) +
  geom_col(fill = "grey30") + coord_flip() +
  labs(x = NULL, y = "Permutation importance", title = "RF variable importance") +
```

```r
  theme_bw(base_size = 10)

p2 <- ggplot(pd_df, aes(years_lived, yhat)) +
  geom_line(linewidth = 1) +
  labs(x = "Years lived at residence", y = "Predicted trust",
       title = "Partial dependence") +
  theme_bw(base_size = 10)

if (requireNamespace("gridExtra", quietly = TRUE)) {
  gridExtra::grid.arrange(p1, p2, ncol = 2)
} else { print(p1); print(p2) }
```

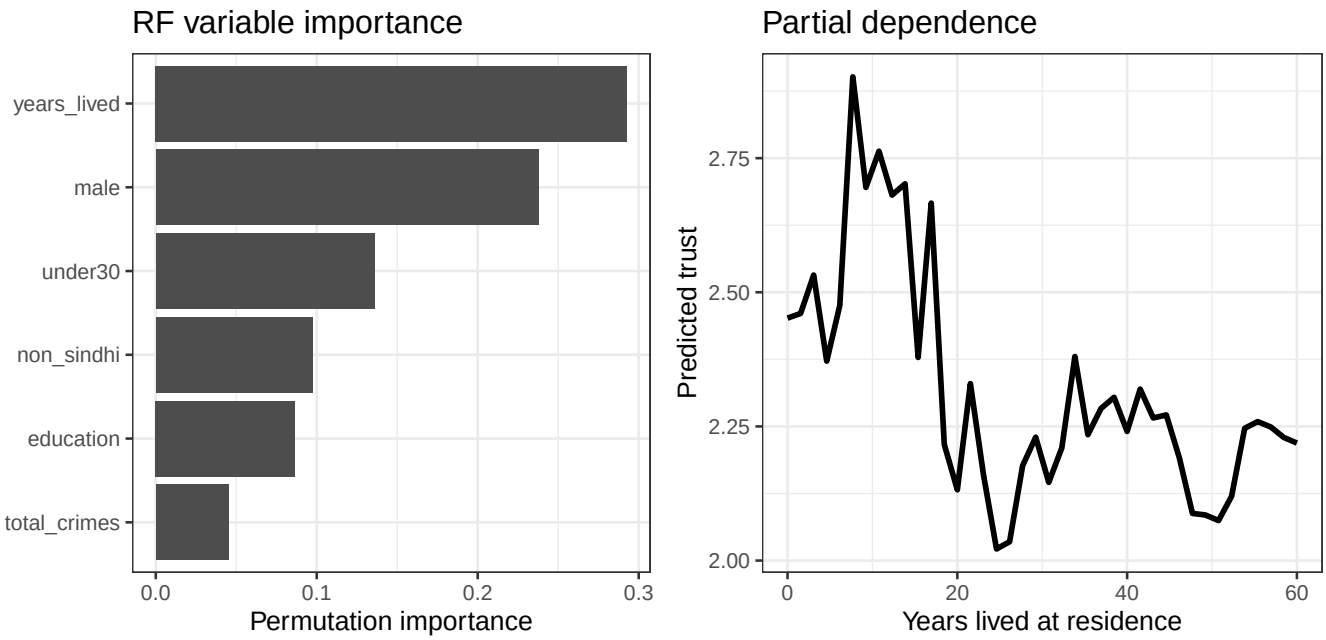

Figure 9: Left: random-forest permutation importance (increase in MSE when a predictor is shuffled). Right: partial dependence of predicted trust on years lived at the current residence, holding other predictors at their observed values.

The permutation importances reorder the story told by the OLS coefficients. In the linear model, `under30` and `non_sindhi` are the largest and most significant terms. The random forest, by contrast, leans heavily on **`years_lived`** and **`male`** — `years_lived` enters the OLS as a small, borderline-significant slope, but the partial-dependence curve shows a clear **nonlinear** pattern (trust falls steeply over the first years of residence, then flattens and can tick back up). This curvature, invisible to the additive linear model, is exactly the structure the tree ensembles use to edge past OLS on held-out data.

12 Takeaway

The published OLS reproduces exactly (`non_sindhi` $= -0.777$, `under30` $= -0.554$) and remains a compelling *explanatory* model: age and Sindhi/non-Sindhi ethnicity are the clearest correlates of trust in Karachi's police. Running the *same* outcome and predictors through Day 3's penalized regressions (ridge, lasso) and the full stable of classic learners — an interpretable `rpart` tree, a `ranger` random forest, `xgboost` boosting, RBF- and linear-kernel SVMs, and CV-tuned k-NN — shows how each learner reaches a prediction differently, yet all plug into one train/test protocol and one scoreboard. On this low-dimensional design the predictive gains over a well-specified OLS are moderate — the forest trims RMSE by about 7% and lifts out-of-sample R^2

from 0.17 to 0.27 — and that *is* the Day-4 lesson: the flexible learners help precisely where the linear model's **functional form** is wrong (the nonlinear `years_lived` effect the partial-dependence plot exposes), and add little where it is right. Interpretation tools — permutation importance and partial dependence — let us see that structure rather than take the ensemble's word for it.

13 Independent exercises (each about 10 minutes)

1. **A second partial-dependence curve.** Reuse the `pd_grid`/`pd_val` pattern with the training-set forest `rf` to sweep `education` instead of `years_lived`. Is its shape closer to the straight line the OLS assumes?
2. **Two importance definitions.** Refit `rf` (same training data) with `importance = "impurity"` and compare the top three predictors with the permutation ranking. Do the two definitions agree?
3. **k and the bias–variance tradeoff.** Re-score k-NN at `k = 5` and `k = 50` (edit the `kknn` call that uses `best_k`) and compare test RMSE. Which failure mode — variance or bias — does each extreme illustrate?